

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 06 -

ARQUITETURAS ESCALÁVEIS NO MUNDO REAL

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	9
O QUE VOCÊ VIU NESTA AULA?	10
REFERÊNCIAS.....	11

EMANIP

O QUE VEM POR AÍ?

Chegamos à nossa última aula. Ao longo desta disciplina, montamos nosso "Lego" da escalabilidade: começamos com os blocos fundamentais, aprendemos a usar LoadBalancers para distribuir o tráfego, aplicamos padrões de arquitetura para resiliência, usamos CDNs para alcançar uma performance global e, por fim, enfrentamos o desafio de escalar o banco de dados. Agora, com todas as peças em mãos, é hora de dar um passo para trás e olharmos para a "planta baixa" completa. Como esses componentes se unem para formar os sistemas robustos que sustentam as maiores empresas de tecnologia do mundo?

Nesta aula final, vamos elevar nossa perspectiva para a de um arquiteto de software. Compararemos os grandes paradigmas que dominam o mercado: as modernas arquiteturas Serverless, o debate entre Microsserviços e Monolitos escaláveis, e os trade-offs de negócio por trás de cada escolha. Vamos aprender a pensar como os engenheiros de empresas como Google, Amazon e Nubank, descobrindo as práticas de observabilidade que os permitem "enxergar" o que está acontecendo em sistemas com milhares de servidores. E, para consolidar todo o nosso aprendizado, aplicaremos nosso conhecimento para resolver um problema de escala do começo ao fim.

HANDS ON

Para nossa última seção prática, faremos um hands-on de arquitetura, o verdadeiro trabalho de um arquiteto de soluções. O objetivo é aplicar todo o conhecimento adquirido na disciplina para analisar um sistema com problemas e propor uma solução escalável, moderna e resiliente. As videoaulas que acompanham este material guiarão você por este estudo de caso completo.

Seremos apresentados a um cenário: uma empresa fictícia de e-commerce com uma arquitetura monolítica tradicional que falhou catastroficamente durante um pico de demanda. Começaremos analisando um diagrama da arquitetura atual. Juntos(as), atuaremos como detetives, identificando os múltiplos pontos de falha e gargalos: a falta de um LoadBalancer, a aplicação "stateful", o banco de dados sobrecarregado e a ausência de uma CDN.

O coração do nosso hands-on será uma sessão de design em um quadro branco digital. A partir da arquitetura problemática, vamos desenhar, componente por componente, a nova solução. Posicionaremos a CDN para servir os assets, introduziremos um ApplicationLoadBalancer para distribuir a carga, colocaremos a aplicação em um Auto ScalingGroup, usaremos uma fila para desacoplar o processamento de pedidos e, por fim, escalaremos o banco de dados com Réplicas de Leitura. Cada peça adicionada será justificada com os conceitos que aprendemos, transformando a teoria em uma proposta de refatoração concreta e defensável.

SAIBA MAIS

Após dominar os componentes individuais da escalabilidade, o próximo passo é entender como eles se encaixam em paradigmas arquiteturais de alto nível. Uma das abordagens mais modernas é a arquitetura serverless. Liderada por serviços como o AWS Lambda, ela representa a máxima abstração da infraestrutura. A pessoa desenvolvedora escreve e envia o código de uma função, e o provedor de nuvem se encarrega de tudo: provisionar, gerenciar, aplicar patches e, o mais importante, escalar. A função pode ser executada centenas de vezes por segundo em resposta a eventos e, quando a demanda cessa, a escala volta a zero e o custo também. É a expressão máxima da escalabilidade automática e da elasticidade.

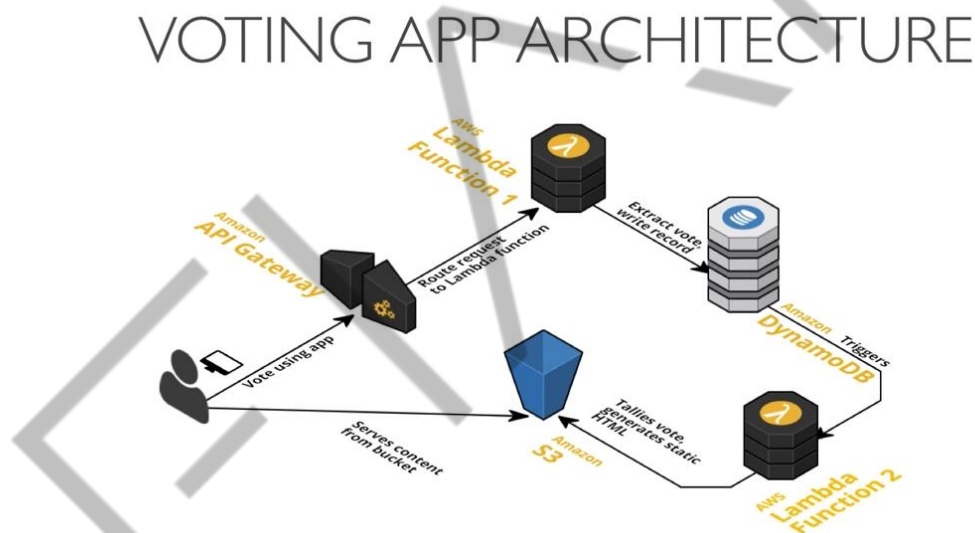


Figura 1 - Exemplo arquitetura Servless com AWS Lambda
Fonte: <https://thorntech.com/serverless-website-aws-lambda/> (2016)

No entanto, a maioria das aplicações ainda reside no debate clássico entre Microsserviços vs. Monolitos Escaláveis. É um erro pensar no monolito como uma arquitetura inerentemente ruim. Um monolito escalável, bem projetado e stateless, pode ser colocado dentro de um Auto ScalingGroup e escalar horizontalmente de forma muito eficaz. Sua principal vantagem é a simplicidade: o desenvolvimento, o teste e o deploy são processos unificados, e a depuração é mais fácil, pois todas as chamadas ocorrem dentro do mesmo processo.

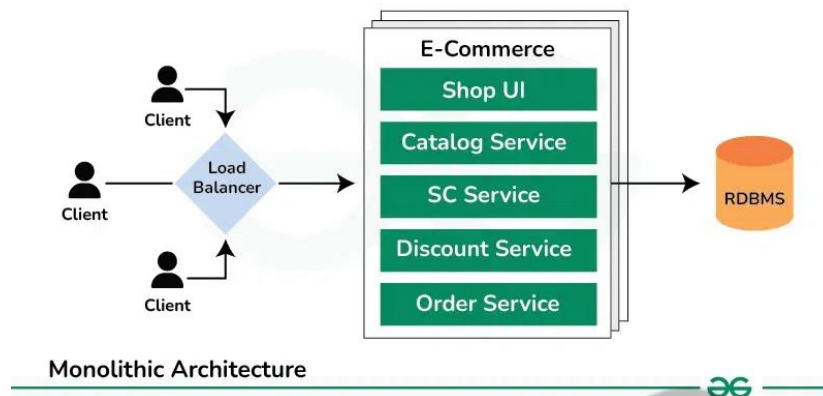


Figura 2 - Arquitetura Monolítica
Fonte: [Geeksforgeeks](#) (2025)

A arquitetura de microsserviços, por outro lado, propõe decompor uma aplicação grande em uma coleção de pequenos serviços independentes, cada um responsável por uma capacidade de negócio específica (ex.: serviço de catálogo, serviço de pagamento). As vantagens são a escalabilidade granular (pode-se escalar apenas o serviço de pagamento, que está sobrecarregado), a independência das equipes (cada time é dono do seu serviço) e a flexibilidade tecnológica (um serviço pode ser em Java, outro em Python). Contudo, essa liberdade vem com um custo: uma complexidade operacional massiva. A comunicação entre serviços pela rede introduz latência e um novo ponto de falha, e a depuração de uma transação que passa por múltiplos serviços se torna um desafio que exige ferramentas especializadas.

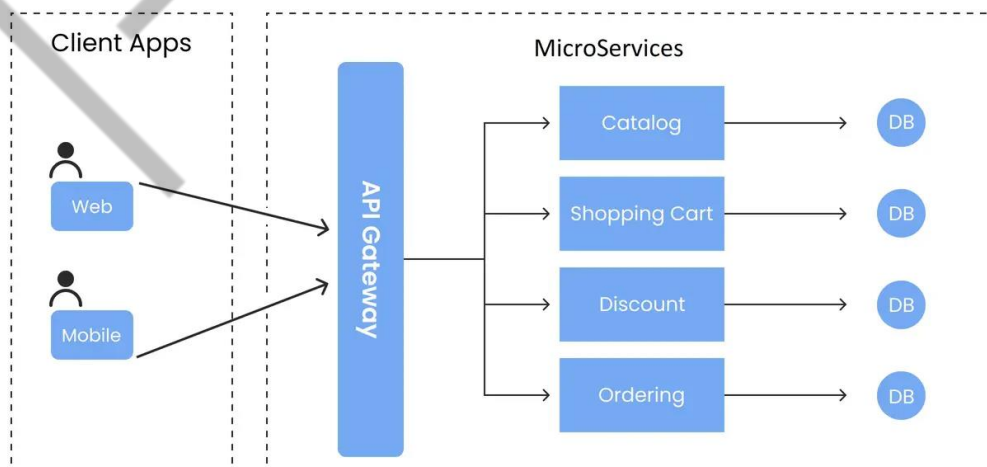


Figura 3 - Arquitetura Microsserviço
Fonte: [Infiniticube](#) (2023)

A escolha entre esses paradigmas envolve profundos trade-offs técnicos e de negócio. A Lei de Conway postula que a arquitetura de um sistema reflete a estrutura de comunicação da organização que o constrói. Pequenas equipes autônomas tendem a produzir microsserviços, enquanto equipes mais centralizadas tendem a produzir monolitos. A decisão, portanto, vai além da tecnologia, impactando a agilidade da empresa e a forma como as equipes colaboram.

Independentemente da arquitetura escolhida, operá-la em larga escala exige uma habilidade fundamental: decidir 'onde está o gargalo?'. Um sistema lento raramente tem uma causa óbvia. O gargalo pode estar na CPU, na memória, no I/O do disco, em uma query de banco de dados, em uma API de terceiro, na latência da rede ou em um cache mal configurado. Para encontrar a resposta, precisamos de observabilidade.

As ferramentas e práticas de observabilidade em ambientes de escala se baseiam em três pilares:

Métricas: dados numéricos agregados ao longo do tempo. São os "sinais vitais" do sistema, como uso de CPU, latência das requisições e taxa de erros. Elas nos dizem o que está acontecendo e nos alertam para problemas.

Logs: registros de eventos discretos e com timestamp. São as "testemunhas oculares" de uma ocorrência. Eles nos dizem por que algo aconteceu, fornecendo o contexto detalhado de um erro ou de uma transação específica.

Traces (Rastreamento Distribuído): mostram a jornada completa de uma única requisição enquanto ela viaja pelos múltiplos serviços em uma arquitetura distribuída. Eles são essenciais para microsserviços e nos dizem onde o tempo foi gasto, permitindo identificar exatamente qual serviço está causando a lentidão.

As grandes empresas que definem o mercado hoje são mestras em escalabilidade e observabilidade. O Google popularizou a cultura de SRE (Site Reliability Engineering), que trata as operações como um problema de software, com foco em automação, orçamentos de erro (error budgets) e métricas de nível de serviço (SLOs). A Amazon opera sob a filosofia de "você constrói, você opera" ("you build it, you run it"), onde cada equipe é dona do seu serviço, incluindo sua operação e escalabilidade, fazendo uso massivo de seus próprios serviços de nuvem. E o Nubank, um dos maiores bancos digitais do mundo, é um caso de sucesso de uma

arquitetura nativa da nuvem, baseada em microsserviços e princípios de programação funcional, onde a observabilidade robusta é a chave para manter a confiabilidade em um ambiente de crescimento explosivo.

Para consolidar todo esse conhecimento, vamos analisar um estudo de caso prático. Imagine um e-commerce com uma arquitetura monolítica clássica: um único servidor web rodando a aplicação e o banco de dados juntos. Na Black Friday, o sistema falhou. Nossa tarefa é propor uma refatoração escalável. Aplicando o que aprendemos, a solução se constrói passo a passo:

- **Aula 4 (CDN):** colocamos uma CDN (CloudFront) na frente para servir assets estáticos (imagens, CSS), aliviando a carga do servidor;
- **Aula 2 (LB):** introduzimos um Application Load Balancer para distribuir o tráfego, eliminando o ponto único de falha;
- **Aula 1 e 3 (Escala e Padrões):** separamos a aplicação do banco de dados. A aplicação, agora stateless, é colocada em um Auto Scaling Group. Processos lentos, como o envio de e-mails, são desacoplados com uma fila (SQS);
- **Aula 5 (Banco de Dados):** o banco de dados é movido para um serviço gerenciado (RDS) e escalamos a leitura com a adição de Read Replicas.

O resultado é a transformação de uma arquitetura frágil em um sistema distribuído, resiliente e escalável, pronto para qualquer pico de demanda.

MERCADO, CASES E TENDÊNCIAS

As práticas de arquitetura e operação de sistemas em larga escala estão em constante evolução, com novas tendências surgindo para lidar com a crescente complexidade dos ambientes em nuvem.

Case: A Cultura de Engenharia do Nubank

O Nubank é um case global de sucesso na construção de uma instituição financeira nativa da nuvem. Em seu blog de engenharia, eles detalham sua arquitetura de microsserviços e a escolha por tecnologias como Clojure e Datomic para garantir consistência em um ambiente distribuído. Eles são um exemplo prático de como a escolha de uma arquitetura moderna e a forte cultura de engenharia, com foco em automação e testes, permitem escalar um negócio complexo e regulado de forma segura e rápida.

Tendência: A Consolidação da Cultura SRE (Site Reliability Engineering)

O que começou como uma prática interna do Google se tornou uma tendência em todo o mercado. Empresas estão cada vez mais abandonando as equipes tradicionais de "Operações" e adotando a cultura SRE. Isso significa tratar as operações como um problema de software, onde as equipes criam automação para substituir tarefas manuais, focam em métricas de confiabilidade (SLOs e Error Budgets) e têm autoridade para priorizar a estabilidade em detrimento de novas funcionalidades quando o "orçamento de erro" é extrapolado.

Tendência: A Ascensão da Engenharia de Plataforma (Platform Engineering)

Com a complexidade das arquiteturas de microsserviços e da nuvem, a carga cognitiva sobre os desenvolvedores de produtos aumentou drasticamente. A Engenharia de Plataforma surge como uma resposta. A ideia é criar equipes de plataforma dedicadas a construir uma "Plataforma de Desenvolvedor Interna" (Internal Developer Platform - IDP). Essa plataforma oferece, como um serviço, tudo o que as equipes de produto precisam para rodar suas aplicações: pipelines de CI/CD, infraestrutura como código, ferramentas de observabilidade e segurança. Isso libera os desenvolvedores para focarem no que eles fazem de melhor: entregar valor de negócio.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula final, unimos todos os pontos e consolidamos nosso conhecimento em uma visão arquitetural completa. Exploramos os grandes **paradigmas arquiteturais**, como **Serverless**, e analisamos os trade-offs entre **Microserviços** e **Monolitos Escaláveis**. Aprendemos sobre a importância da **observabilidade** por meio de seus três pilares, **métricas, logs e traces**, como a principal ferramenta para identificar **gargalos** em produção.

Vimos como as maiores empresas do mundo aplicam esses conceitos e, o mais importante, percorremos um **estudo de caso** completo, aplicando o conhecimento de todo o curso para transformar uma arquitetura frágil em um sistema resiliente e escalável. Você encerra este curso não apenas conhecendo as peças, mas sabendo como combiná-las para construir grandes sistemas.

REFERÊNCIAS

AWS. **AWS Well-Architected Framework**. Amazon Web Services. Disponível em: <https://aws.amazon.com/architecture/well-architected/>. Acesso em: 28 ago. 2025.

BEYER, B. et al. (Eds.). **Site Reliability Engineering: How Google Runs Production Systems**. Sebastopol: O'Reilly Media, 2016.

NEWMAN, S. **Building Microservices: Designing Fine-Grained Systems**. Sebastopol: O'Reilly Media, 2015.

EMANIP

PALAVRAS-CHAVE

Serverless. Monolitico. Microserviço. Observabilidade.

EMENDAS



POSTECH